RepoLens

AI-Powered Codebase Intelligence &
Developer Onboarding Platform

PROJECT CONTINUITY DOCUMENT — UPDATED CURRENT STATE

Purpose: provide a complete, reliable hand-off point for another AI/developer to continue RepoLens without repeating completed work or unnecessarily changing the existing implementation.

Current milestone: GitHub repository metadata retrieval is working end-to-end.

1. Project Vision

RepoLens is a full-stack developer tool designed to help developers understand unfamiliar GitHub repositories. The user provides a GitHub repository URL, and the platform progressively analyzes the repository so that it can explain its structure, technologies, dependencies, architecture and code, answer repository-specific questions, and eventually recommend safe changes.

Core philosophy: build progressively. Do not jump directly into an LLM/RAG implementation. The intended progression is React → FastAPI → GitHub API → repository structure → repository analysis → LLM explanation → embeddings/RAG → architecture intelligence → Change Advisor.

Final product direction: RepoLens should demonstrate both software engineering and applied AI. It should not look like a generic chatbot or CRUD application. Its differentiator is that answers and recommendations are grounded in the actual repository being analyzed. This framing is consistent with the previous project overview. filecite turn7file0 L26-L32

2. Project Objectives

Objective	Planned capability
Repository understanding	Project purpose, technologies, folders, files, dependencies and important components
Code discovery	Find the exact file/function/block responsible for a feature
Code explanation	Explain files and code using repository context
Architecture intelligence	Show frontend/API/service/model/database relationships
Change Advisor	Recommend files/functions to modify and assess potential impact/risk
Developer onboarding	Generate a repository-specific onboarding guide

3. Technology Stack

Area	Technology / status
Frontend	React + Vite + JavaScript + CSS
Routing	React Router
Frontend HTTP	Native browser fetch()
Backend	Python + FastAPI
Server	Uvicorn
Environment	Python virtual environment (.venv)
Configuration	python-dotenv + Backend/.env
GitHub integration	GitHub REST API + authenticated token
Planned AI	LLM + embeddings + vector search + RAG
Planned database	PostgreSQL + SQLAlchemy

4. Current Project Structure

```
RepoLens/
├── .venv/
├── Frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Landing.jsx
│   │   │   ├── Landing.css
│   │   │   ├── Login.jsx
│   │   │   ├── Login.css
│   │   │   ├── Signup.jsx
│   │   │   └── Signup.css
│   │   ├── App.jsx
│   │   └── ...
│   ├── package.json
│   └── ...
├── Backend/
│   ├── main.py
│   └── .env
└── ...
```

Security: Backend/.env contains the GitHub token and must never be committed or copied into documentation. Add .env to .gitignore.

5. Frontend Status

Landing page: completed. It contains the RepoLens brand, Login/Sign Up navigation, hero section, repository URL input, Analyze Repository button, animated pipeline, repository result display, core capability cards, workflow section and footer.

Login: UI completed. It currently contains email/password fields, forgot-password link, sign-in button, account-creation navigation and GitHub button. Full authentication is not implemented yet.

Signup: UI completed. It contains name, email, password and confirm-password fields plus account creation/login navigation. Full backend authentication is not implemented yet.

React concepts already in use: reusable components, JSX, className, useState, Link, Routes/Route, onChange and fetch(). The previous overview records these as completed concepts. □filecite□turn7file1□L62-L72□

6. React Routing

```
/
/login
/signup
```

Current App.jsx maps these three routes to Landing, Login and Signup. Do not change these routes unless a future feature actually requires it.

7. Current Landing.jsx Integration

Landing.jsx currently maintains:

```
const [repoUrl, setRepoUrl] = useState("");
const [response, setResponse] = useState(null);
```

The Analyze button validates that a URL was entered, sends it to FastAPI using native fetch(), reads the JSON response, stores it in response state, and displays either an error or repository metadata.

```
POST http://127.0.0.1:8000/analyze?repo_url=<encoded_repo_url>
```

The repository result was moved outside the input container so the result card is displayed as a separate block underneath the input/button row. The old “No repository connected yet” message is shown only when there is no response.

8. Backend Status

The backend environment, FastAPI, Uvicorn and CORS setup are complete. The earlier project document recorded the Python environment, FastAPI, Uvicorn, GET /, POST /analyze and CORS as completed foundations. filecite turn7file1 L73-L82

```
GET /
→ {"message": "RepoLens backend is running"}

POST /analyze
→ accepts repo_url
→ validates GitHub URL
→ calls GitHub REST API
→ returns selected repository metadata
```

CORS currently allows the Vite development server:

```
http://localhost:5173
```

9. GitHub Token & API Integration

Today the project moved beyond the original placeholder /analyze response. A GitHub personal access token was generated and stored in Backend/.env. python-dotenv was installed and used to load the token.

```
GITHUB_TOKEN=YOUR_ACTUAL_TOKEN
```

The actual token must never be placed in source code, screenshots, documentation, GitHub commits or messages.

The backend authenticates requests to the GitHub REST API using the token. The repository endpoint is conceptually:

```
https://api.github.com/repos/{owner}/{repo}
```

The backend extracts the owner/repository name from the supplied GitHub URL and retrieves real repository metadata.

10. Current Repository Metadata

The current backend returns selected metadata such as:

```
{
  "name": "react",
  "owner": "react",
  "description": "The library for web and native user interfaces.",
  "stars": 247421,
  "forks": 51243,
  "language": "JavaScript",
  "default_branch": "main",
  "github_url": "..."
}
```

Exact values are dynamic because GitHub repository statistics change. The fields currently surfaced by the UI are repository name, description, owner, language, stars, forks and default branch.

11. Error Handling

The backend handles invalid GitHub URLs/repository input and GitHub/network errors. The frontend handles empty input, backend/API errors and connection failures. This keeps the current metadata milestone usable before more advanced analysis is added.

12. Testing Completed Today

A public repository such as:

```
https://github.com/facebook/react
```

was used to test the complete integration. The important successful flow is:

```
Browser / React
      ↓ fetch()
POST /analyze
      ↓
FastAPI / Uvicorn
      ↓ authenticated GitHub REST API
GitHub repository metadata
      ↓ JSON
React response state
      ↓
Repository result card
```

The backend returned a successful request, and the real metadata reached the React UI. The previous project document already established the React → FastAPI test flow; today's work completed the GitHub metadata portion of that milestone. filecite turn7file6 L463-L490

13. Current Local Development URLs

Service	Local URL
React/Vite	http://localhost:5173
FastAPI	http://127.0.0.1:8000
FastAPI docs	http://127.0.0.1:8000/docs

14. Current CSS Design System

The existing Landing.css uses a dark developer-tool aesthetic and should be preserved. The project overview defines the following visual system:

Token	Value
Main background	#090c12
Secondary background	#0d1119
Surface	#11151f
Surface hover	#151b27
Main border	#1f2632
Soft border	#171d29
Primary text	#e7ecf6
Secondary text	#a3adbf
Muted text	#8993a8
Faint text	#5b647a
Accent A	#7c9cff
Accent B	#4fd1c5
Display font	Space Grotesk
Body/UI font	Inter
Technical/code font	JetBrains Mono

The primary accent gradient is #7C9CFF → #4FD1C5. Hero backgrounds use subtle radial gradients, cards use dark linear gradients with thin borders, and unrelated bright colors should be avoided except where semantic status requires them. filecite turn7file2 L93-L109

15. Existing Landing CSS Sections

- Main page
- Navbar
- Hero section
- Pipeline animation
- Hero glow
- Repository input
- Hero flow
- Feature cards
- How It Works
- Footer
- Tablet responsiveness
- Mobile responsiveness

The existing SVG PipelineTrace animation visually communicates Code → Repository → AI → Understanding. Its design should not be removed or replaced unnecessarily.

16. Repository Result CSS Added Today

```
.repository-result {
  width: min(700px, 100%);
  margin: 28px auto 0;
  padding: 24px;
  box-sizing: border-box;
  border: 1px solid rgba(124, 156, 255, 0.25);
  border-radius: 16px;
  background: linear-gradient(
    145deg,
    rgba(17, 21, 31, 0.96),
    rgba(12, 16, 24, 0.96)
  );
  text-align: center;
}

.repository-result h3 {
  margin: 0 0 10px;
  font-size: 24px;
}

.repository-result p {
  margin: 8px 0;
  color: var(--text-secondary);
  font-size: 13px;
  line-height: 1.6;
}

.repository-details {
  display: flex;
  align-items: center;
  justify-content: center;
  flex-wrap: wrap;
  gap: 18px;
  margin: 18px 0;
}
```

```
.repository-details span {
  color: var(--text-muted);
  font-size: 12px;
  white-space: nowrap;
}
```

Mobile rules were also added so the repository result remains responsive. Existing Landing.css styling was preserved; only the new repository-result classes were introduced.

17. Current Product Flow

```
User
 ↓
Landing page
 ↓
Paste GitHub repository URL
 ↓
Analyze Repository
 ↓
React fetch()
 ↓
FastAPI /analyze
 ↓
GitHub REST API
 ↓
Repository metadata
 ↓
FastAPI JSON
 ↓
React
 ↓
Repository result card
```

18. Current Completion Status

Feature	Status
React + Vite setup	COMPLETED
Login UI	COMPLETED
Signup UI	COMPLETED
Login ↔ Signup routing	COMPLETED
Landing page UI	COMPLETED
Landing → Login/Signup navigation	COMPLETED
Python virtual environment	COMPLETED
FastAPI setup	COMPLETED
Uvicorn setup	COMPLETED
CORS configuration	COMPLETED
React → FastAPI communication	COMPLETED
GitHub URL → /analyze	WORKING
GitHub authenticated metadata retrieval	WORKING
Metadata display in React	WORKING
Real user authentication	NOT STARTED

Feature	Status
Repository file/folder tree	NEXT
Repository analysis	FUTURE
AI/RAG	FUTURE

19. Major Project Scope

RepoLens is suitable as a major project if the roadmap is implemented beyond metadata retrieval. Its intended scope combines full-stack development, GitHub repository analysis, software-engineering intelligence, LLM-based explanation, RAG, architecture analysis and change-impact guidance.

The project should not stop at “paste repository → chatbot answer.” The important academic/technical value comes from building structured repository understanding before using AI.

20. Full Future Roadmap

Phase	Main objective
1	Frontend foundation: React, Vite, routing, Login, Signup, Landing
2	React ↔ FastAPI communication and GitHub repository import — current milestone completed
3	Repository structure analysis: files, folders, languages, dependencies, README and configuration
4	LLM-powered project overview and file/folder explanations
5	Code chunking + embeddings + vector search + RAG for repository-aware Q&A
6	Architecture intelligence and visualization of frontend/API/service/database relationships
7	Change Advisor: relevant files, functions, dependencies, likely changes and risk
8	Repository-specific developer onboarding guide
9	Advanced features: risky files, contribution suggestions, dependency analysis, code quality, test coverage, PR

21. Phase 3 — Immediate Next Step: Repository Structure

The next implementation should retrieve the repository file/folder structure. This is the immediate milestone because repository metadata alone cannot explain how the codebase is organized.

Recommended first implementation:

```
GitHub URL
   ↓
Repository metadata          DONE
   ↓
Repository contents/tree      NEXT
   ↓
File paths + folder paths
   ↓
Technology/dependency detection
   ↓
README/configuration analysis
   ↓
Repository overview
```

Start with the smallest working version. Explain GitHub's repository contents API, add a clean FastAPI endpoint, return structured file/folder data, connect it to React, and test it using facebook/react.

22. Phase 3 Details — What to Retrieve

- Root folders and files
- Nested folders and files
- File paths and extensions
- README.md
- package.json
- requirements.txt / pyproject.toml
- Dockerfile and relevant configuration
- Other dependency manifests
- Project entry points
- Technology/framework indicators

Never retrieve or expose actual secret files such as repository .env contents.

23. Phase 4 — LLM Project Overview

Only after structured repository information is reliable should an LLM be introduced. The LLM should receive repository-derived context rather than only the raw GitHub URL.

- Project summary
- Architecture explanation
- Technology explanation
- Important component explanation
- Repository onboarding summary

24. Phase 5 — Embeddings + RAG

```
Repository files
      ↓
File filtering
      ↓
Code chunking
      ↓
Embeddings
      ↓
Vector database
      ↓
Similarity search
      ↓
Relevant repository context
      ↓
LLM
      ↓
Repository-aware answer
```

This is the stage that makes RepoLens more than a generic chatbot. The RAG system must retrieve relevant repository code before the LLM generates an answer. □filecite□turn7file5□L226-L250□

25. Phase 6 — Architecture Intelligence

Build relationships such as:

```
Frontend
   ↓
API
   ↓
Services
   ↓
Models
   ↓
Database
```

- Architecture diagram
- Dependency relationships
- Service relationships
- API-to-file mapping
- Frontend-to-backend mapping

26. Phase 7 — Change Advisor

Example request:

```
Add Google login.
```

RepoLens should eventually identify:

- Likely frontend files
- Likely backend files
- Authentication service
- Database/model changes
- Configuration changes
- Required dependencies
- Potential risks

The system should explain why each file matters. This is one of the strongest differentiating features in the planned product.

27. Phase 8 — Developer Onboarding

Generate a repository-specific guide containing:

- Repository overview
- How to run locally
- Architecture
- Important directories
- Important files
- Request flow
- Database flow
- Authentication flow
- Common development tasks
- Where to make changes

28. Phase 9 — Advanced Features

- Risky-file detection
- Contribution suggestions
- Dependency analysis
- Code quality analysis
- Test coverage analysis
- Pull request analysis
- Repository health score
- Change impact analysis
- Security-oriented repository checks

These features should come after the core repository analysis and AI workflow are reliable.
filecite turn7file5 L333-L347

29. Authentication Roadmap

Current Login and Signup are UI-only. Full authentication should be introduced after the core repository-analysis workflow is stable unless a specific feature requires it.

```
Frontend Login/Signup
        ↓
FastAPI authentication
        ↓
PostgreSQL
        ↓
JWT
```

GitHub OAuth can be added later.

30. Database Roadmap

```
PostgreSQL
    +
SQLAlchemy
```

Potential future entities:

```
User
Repository
RepositoryAnalysis
File
AnalysisSession
ChatSession
Message
```

Do not introduce the database prematurely if it is not needed for the immediate repository-analysis milestone.

31. Deployment Roadmap

```
React/Vite frontend
        ↓
Frontend hosting
        ↓
FastAPI backend
        ↓
Backend hosting
        ↓
PostgreSQL cloud database
```

- Replace localhost CORS origin

- Configure production environment variables
- Never expose the GitHub token
- Configure production API URL
- Secure backend endpoints
- Verify GitHub API rate limits

32. Rules for Any AI Continuing RepoLens

- Treat this document as the current project source of truth.
- Do not redo completed setup.
- Do not recreate the GitHub token workflow unless it is broken.
- Do not unnecessarily rewrite Landing.jsx, Landing.css, Login, Signup or App.jsx.
- Preserve the RepoLens name and current dark developer-tool design.
- Preserve #7C9CFF → #4FD1C5 as the main accent gradient.
- Preserve current React Router paths: /, /login, /signup.
- Build incrementally and test every milestone.
- Explain unfamiliar concepts before introducing them.
- Do not expose secrets from .env or analyzed repositories.
- Do not jump to LLM/RAG before repository structure and analysis are reliable.
- When modifying existing code, change only what is required unless a redesign is explicitly requested.

33. Exact Starting Point for the Next AI

> RepoLens frontend and backend setup are complete. React Router, Landing/Login/Signup UI, Python virtual environment, FastAPI, Uvicorn and CORS are working. The Landing page sends a GitHub repository URL to POST /analyze using native fetch(). Backend/.env contains a GitHub token loaded with python-dotenv. FastAPI authenticates with the GitHub REST API and successfully retrieves real repository metadata. React displays that metadata in a styled repository-result card. Current local servers are http://localhost:5173 and http://127.0.0.1:8000. Do not redo this work. The next task is repository structure/file retrieval, followed by technology/dependency/README analysis. Only after structured repository analysis is stable should LLM, embeddings and RAG be introduced.

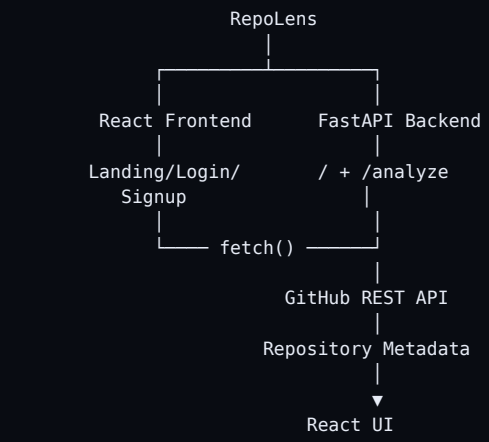
This exact continuation point is consistent with the prior project continuity document.
□filecite□turn7file8□L565-L619□

34. Immediate Next Session Checklist

Step	Action
1	Explain GitHub repository contents API
2	Design the repository structure endpoint
3	Implement the smallest working FastAPI endpoint
4	Retrieve folders/files for a public repository
5	Return clean structured JSON

Step	Action
6	Connect React to the new endpoint
7	Display repository tree in the existing UI style
8	Test using https://github.com/facebook/react
9	Only after success, add recursive/deeper analysis

35. Current End-to-End State




```
NEXT:
Repository file/folder tree
      ↓
Technology + dependency analysis
      ↓
Repository overview
      ↓
LLM explanation
      ↓
RAG
      ↓
Architecture intelligence
      ↓
Change Advisor
      ↓
Developer onboarding
```

36. Final Project Description

> RepoLens — AI-Powered Codebase Intelligence & Developer Onboarding Platform
>
> A full-stack platform that analyzes GitHub repositories, explains codebase architecture, enables repository-aware AI Q&A;, generates developer onboarding guides, and recommends code changes for new features.

The finished project should demonstrate software engineering and applied AI while remaining grounded in real repository data. This is the intended final direction in the previous project overview.
filecite turn7file7 L513-L524